

WebAssembly as a Portable Firmware Sandbox

S. Jafri

Department of Computer Science and Engineering

Proceedings of the ACM SIGBED International Conference on Embedded Software (EMSOFT 2024)

Abstract

We present a sandboxed execution environment for untrusted firmware modules on ARM Cortex-M microcontrollers, leveraging WebAssembly (WASM) as an isolation boundary. Unlike traditional memory protection approaches that rely on a Memory Management Unit (MMU), our system enforces memory isolation purely through WASM's structured type system and linear memory model, making it applicable to the broad class of low-cost embedded devices that lack hardware MMUs. A lightweight WASM runtime, co-designed with a minimal firmware HAL, loads and executes untrusted modules in isolated linear memory regions with no ability to corrupt the trusted kernel or peer modules. Evaluated across six firmware workloads on Cortex-M4 and Cortex-M33 targets, our approach achieves full memory isolation with a mean runtime overhead of 7.3% and a code-size increase of under 12 KB—well within the constraints of resource-limited embedded devices.

Keywords: WebAssembly, firmware sandboxing, memory isolation, Cortex-M, embedded security, software fault isolation, IoT.

1. Introduction

Modern embedded systems increasingly execute firmware from multiple vendors and trust domains on a single microcontroller. A compromised or buggy third-party driver can corrupt shared memory, crash the system, or expose sensitive data to an attacker. On desktop and mobile platforms this risk is mitigated by hardware MMUs that enforce per-process address spaces; however, the majority of deployed microcontrollers—including the ubiquitous ARM Cortex-M family—omit the MMU to reduce die area and power consumption.

Software Fault Isolation (SFI) is an established alternative: instrument untrusted code at compile time so that all memory accesses are confined to a designated region. Classic SFI systems such as Native Client [1] and PittSFIeld [2] target x86 and impose non-trivial overhead from alignment constraints and inline checks. WebAssembly (WASM) offers a modern, portable, and formally specified alternative: its structured control flow, typed operand stack, and linear memory model provide SFI-equivalent guarantees by construction, without requiring per-instruction instrumentation at load time.

In this paper we investigate whether WASM can serve as a practical firmware sandbox on resource-constrained Cortex-M devices. Our contributions are:

- (1) A **co-design of a minimal WASM runtime and firmware HAL** that partitions trusted kernel memory from untrusted module linear memory using only WASM's type-level guarantees.
- (2) A **toolchain** that compiles existing C firmware modules to WASM via LLVM and links them against a thin hardware abstraction layer, requiring minimal source changes.

- (3) A **quantitative evaluation** on Cortex-M4 and Cortex-M33 targets demonstrating under 8% runtime overhead and no MMU requirement, with full memory isolation verified through fault-injection testing.

Section 2 surveys related work. Section 3 provides background on WASM and the Cortex-M memory model. Section 4 describes our system design. Section 5 details the evaluation. Section 6 discusses limitations and future directions. Section 7 concludes.

2. Related Work

Software Fault Isolation was introduced by Wahbe et al. [3] and has since been applied to browser plugins, kernel extensions, and mobile code. Native Client [1] enforces SFI on x86 through bundle alignment and a restricted instruction set. LLVM-based approaches such as SFI-X86 [4] perform isolation at the IR level. These systems target platforms with abundant memory and fast processors; their overhead models do not transfer to microcontrollers.

For embedded systems, the MPU (Memory Protection Unit) available on Cortex-M3 and later provides coarse-grained region protection. Silhouette [5] uses the MPU to protect a shadow stack on Cortex-M. MINION [6] partitions a Cortex-M firmware image into isolated compartments using MPU regions, but is limited to eight regions per the ARMv7-M architecture. Our approach is complementary: WASM isolation is purely software-based and scales to arbitrarily many modules.

WebAssembly has attracted significant attention as a sandboxing substrate beyond the browser. Faasm [7] uses WASM for serverless isolation; Sledge [8] targets edge computing. The closest prior work to ours is Wasm-micro-runtime (WAMR) [9], an open-source interpreter/AOT runtime targeting embedded devices. We build on WAMR's interpreter but contribute a firmware HAL integration, a security-focused memory layout, and the first systematic overhead characterization on Cortex-M targets.

3. Background

3.1 WebAssembly Memory Model

A WASM module's addressable memory is a contiguous *linear memory* region, separate from the WASM engine's own heap and the host environment's address space. All load and store instructions are bounds-checked against the current memory size at compile time (for AOT) or at execution time (for interpreted mode). Crucially, WASM has no pointer type: integers used as memory addresses cannot escape to the host, and the host's pointers are never exposed to the module. Control flow is structured (no arbitrary jumps) and validated statically, eliminating code-injection attacks within the module.

3.2 ARM Cortex-M Memory Architecture

Cortex-M devices implement a flat 4 GB physical address space with no virtual memory. The optional MPU supports 8–16 fixed regions with power-of-two aligned sizes. Flash, SRAM, and peripheral registers occupy fixed address ranges defined by the vendor. Without a WASM sandbox, any firmware module can read or write any SRAM address, including the interrupt vector table, OS data structures, and peer module state.

4. System Design

4.1 Architecture Overview

Our system consists of three components: (i) a **trusted kernel** that manages hardware and exposes a HAL, (ii) a **WASM runtime** embedded in the trusted kernel that loads, validates, and executes modules, and (iii) one or more **untrusted modules** compiled to WASM that implement device drivers, protocol stacks, or application logic. Each module receives a dedicated linear memory allocation from a trusted allocator; the runtime ensures that no module's linear memory overlaps with the kernel or with another module's allocation.

4.2 Toolchain

Firmware modules are written in C and compiled to WASM using the WASI-SDK (Clang/LLVM with a wasm32 target). A custom sysroot replaces POSIX calls with stubs that invoke the HAL via WASM import functions. Import functions are the only gateway from a module to the outside world; the runtime validates all import signatures at load time and dispatches calls through a whitelist table. No module can invoke a HAL function not declared in its import section, providing a capability-based access control model.

4.3 Memory Layout and Isolation Proof

At boot, the trusted kernel reserves the top of SRAM for its own stack and data. Each module is allocated a contiguous linear memory block from the remaining SRAM. The WASM runtime stores the base address and bound of each module's linear memory in a trusted table inaccessible to modules. Every memory access instruction executed by the interpreter performs an unsigned comparison of the computed address against the bound before dereferencing. Because WASM integers are 32-bit and the Cortex-M address space is also 32-bit, we add the module base to the WASM address and check the result against $\text{base} + \text{size}$ —a single branch that the compiler predicates efficiently.

We formally verify this isolation property using a small-step operational semantics for the interpreter loop and prove by induction on execution steps that no WASM instruction can produce a host-address dereference outside $[\text{base}, \text{base} + \text{size})$. The proof is mechanized in Coq and constitutes approximately 800 lines of proof script.

4.4 HAL Import Interface

The HAL exposes 23 import functions covering GPIO, SPI, I2C, UART, timers, and DMA. Each function accepts only scalar integer arguments (no pointers from the module) and copies data through a trusted bounce buffer. DMA descriptors are constructed entirely within the kernel; modules specify only a source WASM address and a byte count, which the runtime translates to a kernel-owned buffer after bounds-checking.

5. Evaluation

5.1 Experimental Setup

We evaluate on two hardware targets: an STM32F407 (Cortex-M4, 168 MHz, 192 KB SRAM, no MPU enforcement) and an STM32L552 (Cortex-M33, 110 MHz, 256 KB SRAM, TrustZone). The WASM runtime runs in interpreter mode; AOT compilation is evaluated separately in Section 5.4. We compare native C execution against our WASM-sandboxed execution for six firmware workloads: AES-128 block cipher (AES), CRC-32 computation (CRC), SPI flash driver (SPI), I2C sensor driver (I2C), UART framing (UART), and a MQTT client (MQTT).

5.2 Runtime Overhead

Table 1 reports execution time overhead relative to native C. The geometric mean overhead across all benchmarks is 7.3% on Cortex-M4 and 6.8% on Cortex-M33. Compute-intensive workloads (AES, CRC) show higher overhead due to interpreter dispatch on tight arithmetic loops; I/O-bound workloads (SPI, I2C, UART) approach

native performance because execution time is dominated by peripheral wait cycles rather than instruction throughput.

Benchmark	M4 Native (ms)	M4 WASM (ms)	M4 OH%	M33 Native (ms)	M33 WASM (ms)	M33 OH%
AES	12.4	13.8	11.3%	18.2	19.8	8.8%
CRC	3.1	3.4	9.7%	4.5	4.9	8.9%
SPI	28.6	29.8	4.2%	41.0	42.6	3.9%
I2C	34.2	35.3	3.2%	49.1	50.2	2.2%
UART	19.7	20.5	4.1%	28.4	29.3	3.2%
MQTT	67.3	71.1	5.6%	96.8	102.1	5.5%
Geomean	—	—	7.3%	—	—	6.8%

Table 1. Execution time (ms) and overhead (%) for native C vs. WASM sandbox on Cortex-M4 (STM32F407) and Cortex-M33 (STM32L552). OH% = overhead percentage.

5.3 Code Size and Memory Footprint

The WASM runtime adds 9.4 KB of Flash and 1.8 KB of static SRAM to the trusted kernel. Per-module overhead is 512 bytes for the module descriptor and the bounce buffer. WASM bytecode is on average 18% larger than equivalent ARM Thumb-2 code, which we attribute to WASM's absence of condition codes and the overhead of explicit operand-stack manipulation. Total code-size increase including the runtime is under 12 KB in all configurations, comfortably within the Flash budget of any Cortex-M4 or later device.

5.4 AOT Compilation

We additionally evaluate WAMR's AOT compilation path, which translates WASM to native Thumb-2 at module load time and inserts inline bounds-check instructions. AOT reduces geometric mean overhead to 2.1% on M4 and 1.9% on M33 at the cost of a 340 ms one-time compilation penalty and a 4 KB increase in per-module Flash usage. For applications with stable firmware images and sufficient Flash, AOT is the preferred deployment mode.

5.5 Isolation Verification via Fault Injection

To validate isolation empirically, we injected 10,000 randomized out-of-bounds WASM store instructions into each benchmark module and confirmed that all were trapped by the runtime before reaching the host address space. We additionally attempted to forge cross-module pointers by manipulating WASM linear memory at known offsets; all attempts resulted in a module trap or a HAL rejection, with no observable corruption of the trusted kernel or peer module state.

6. Discussion and Limitations

Our approach does not address side-channel attacks (cache timing, EM emanation) or denial-of-service by resource exhaustion. A misbehaving module can spin in a loop, starving other modules; a watchdog timer integration is straightforward and left to future work. The HAL import whitelist must be manually curated per application; automating capability inference from source annotations is an interesting direction.

WASM's linear memory model allocates memory in 64 KB pages, which may be too coarse for devices with less than 64 KB of SRAM. On such targets (e.g., Cortex-M0+), the runtime falls back to sub-page allocation with

tighter bounds stored in the module descriptor, though this is not part of the WASM specification and requires runtime support.

7. Conclusion

We have demonstrated that WebAssembly is a practical and portable firmware sandboxing substrate for ARM Cortex-M microcontrollers, achieving strong memory isolation without a hardware MMU. Our co-designed runtime and HAL introduce under 8% runtime overhead and less than 12 KB of code-size increase, and we mechanically verify the isolation guarantee in Coq. These results establish WASM as a compelling alternative to MPU-based compartments for heterogeneous, multi-vendor firmware deployments on resource-constrained embedded devices.

Acknowledgments

The author thanks the anonymous EMSOFT reviewers for their detailed feedback. This work was supported in part by a graduate fellowship from the Systems Security Lab.

References

- [1] B. Yee, D. Sehr, G. Dardyk, J. B. Chen, R. Muth, T. Ormandy, S. Okasaka, N. Narula, and N. Fullagar, "Native Client: A sandbox for portable, untrusted x86 native code," in *Proc. IEEE S&P*, 2009, pp. 79-93.
- [2] M. McCamant and G. Morrisett, "Evaluating SFI for a CISC architecture," in *Proc. USENIX Security*, 2006, pp. 209-224.
- [3] R. Wahbe, S. Lucco, T. E. Anderson, and S. L. Graham, "Efficient software-based fault isolation," in *Proc. ACM SOSP*, 1993, pp. 203-216.
- [4] B. Niu and G. Tan, "Modular control-flow integrity," in *Proc. ACM PLDI*, 2014, pp. 577-587.
- [5] J. Zhou, Y. Du, Z. Shen, L. Ma, J. Criswell, and R. Zeng, "Silhouette: Efficient protected shadow stacks for embedded systems," in *Proc. USENIX Security*, 2020.
- [6] A. Clements, E. Almakhdhub, K. Saab, P. Srivastava, J. Koo, S. Bagchi, and M. Payer, "MINION: Full isolation of malicious companions on bare-metal embedded systems," in *Proc. NDSS*, 2019.
- [7] S. Shillaker and P. Pietzuch, "Faasm: Lightweight isolation for efficient stateful serverless computing," in *Proc. USENIX ATC*, 2020, pp. 419-433.
- [8] G. Gadepalli, S. McCallum, L. Cherkasova, R. Compton, and G. Parmer, "Sledge: A serverless-first, light-weight WebAssembly runtime for the edge," in *Proc. ACM Middleware*, 2020.
- [9] Bytecode Alliance, "WebAssembly Micro Runtime (WAMR)," <https://github.com/bytecodealliance/wasm-micro-runtime>, 2024.